In [2]:
```
import numpy as np
import pandas as pd
from scipy import stats
```

In [3]:
```
!pip install scipy
```

```
Requirement already satisfied: scipy in C:\Users\arish\OneDrive\Desktop\CSE\CSE-1
1\HCL\Batch1\myenv\Lib\site-packages (1.18.1)
Requirement already satisfied: numpy<2.8,>=2.0.0 in C:\Users\arish\OneDrive\Deskt
op\CSE\CSE-11\HCL\Batch1\myenv\Lib\site-packages (from scipy) (2.5.4)
```

In [4]:
```
np.random.seed(42)
n = 40
df = pd.DataFrame({
    "Student_ID": range(1, n + 1),
    "Branch": np.random.choice(["CSE", "ECE", "ME", "CE"], n),
    "Study_Hours": np.round(np.random.uniform(1, 8, n), 1),
    "Attendance": np.random.randint(55, 100, n),
})
df["Marks"] = np.clip(
    np.round(30 + 5 * df["Study_Hours"] + 0.2 * df["Attendance"] + np.random.nor
    0, 100
).astype(int)

df.head()
```

Out[4]:

	Student_ID	Branch	Study_Hours	Attendance	Marks
0	1	ME	5.3	70	75
1	2	CE	2.0	99	68
2	3	CSE	3.0	72	59
3	4	ME	3.6	78	70
4	5	ME	4.2	80	69

Task 1: Create a function to assign grades based on marks

- > Write a function get_grade(marks) that returns "A" for 75 and above, "B" for 60 to 74, "C" for 40 to 59, and "F" below 40.
- > Add a Grade column to df using apply().
- > Show how many students got each grade using value_counts().

In [5]:
```
def get_grade(marks):
    if marks >= 75:
        return "A"
    elif marks >= 60:
        return "B"
```

```
    elif marks >= 40:
        return "C"
    else:
        return "F"

# Apply the grading function to create a new Grade column
df["Grade"] = df["Marks"].apply(get_grade)

# Count the number of students in each grade category
grade_counts = df["Grade"].value_counts()

print("Grade Counts:")
print(grade_counts)

# Total number of students
print("\nTotal Students:", grade_counts.sum())

# Display first 5 rows
df.head()
```

```
Grade Counts:
Grade
B    16
A    12
C    12
Name: count, dtype: int64

Total Students: 40
```

Out[5]:

	Student_ID	Branch	Study_Hours	Attendance	Marks	Grade
0	1	ME	5.3	70	75	A
1	2	CE	2.0	99	68	B
2	3	CSE	3.0	72	59	C
3	4	ME	3.6	78	70	B
4	5	ME	4.2	80	69	B

Task 2: Create a list of marks for students with attendance below 75

- > Using a list comprehension, create a list of marks of students whose attendance is below 75.
- > Using filter() and a lambda, get all marks above 70.
- > Using map() and a lambda, add 5 grace marks to every student's marks (the maximum must stay 100).

In [6]:

```
low_attendance_marks = [
    marks
    for marks, attendance in zip(df["Marks"], df["Attendance"])
    if attendance < 75
```

```
]

print("Marks of students with attendance below 75:")
print(low_attendance_marks)


# Use filter() and lambda to get all marks above 70
marks_above_70 = list(filter(lambda x: x > 70, df["Marks"]))

print("\nMarks above 70:")
print(marks_above_70)


# Use map() and lambda to add 5 grace marks (maximum 100)
grace_marks = list(map(lambda x: min(x + 5, 100), df["Marks"]))

print("\nMarks after adding 5 grace marks:")
print(grace_marks)
```

```
Marks of students with attendance below 75:
[75, 59, 60, 57, 78, 82, 55, 80, 48, 64, 54, 58, 65, 82, 80, 69]

Marks above 70:
[75, 74, 81, 72, 78, 82, 80, 87, 78, 78, 83, 82, 80, 84]

Marks after adding 5 grace marks:
[80, 73, 64, 75, 74, 79, 69, 86, 77, 65, 66, 62, 59, 83, 87, 72, 60, 65, 85, 68,
53, 69, 59, 92, 64, 83, 63, 83, 73, 64, 88, 70, 87, 85, 74, 89, 52, 60, 55, 65]
```

Task 3: Flexible functions

- > Write a function summary(*values, round_to=2) that accepts any number of values and returns a dictionary with the count, minimum, maximum and average.
- > Test it with summary(10, 20, 30) and with summary(*df["Marks"]).

In [7]:

```
def summary(*values, round_to=2):
    return {
        "count": len(values),
        "minimum": min(values),
        "maximum": max(values),
        "average": round(sum(values) / len(values), round_to)
    }

# Test the function with sample values
print(summary(10, 20, 30))

# Test the function with all marks from the DataFrame
print(summary(*df["Marks"]))
```

```
{'count': 3, 'minimum': 10, 'maximum': 30, 'average': 20.0}
{'count': 40, 'minimum': 47, 'maximum': 87, 'average': 66.92}
```

Task 4: Save the DataFrame to a CSV file

- Save df to a file named students.csv using to_csv(index=False).
- Write a function load_data(filename) that reads a CSV file using try and except. If the file is not found, it should print a friendly message instead of crashing.
- Test it once with the correct file name and once with a wrong name.

In [8]:
```
df.to_csv("students.csv", index=False)

print("students.csv saved successfully!")


# Function to safely load data from a CSV file
def load_data(filename):
    try:
        data = pd.read_csv(filename)
        print(f"\n{filename} loaded successfully!")
        print(data.head())
        return data
    except FileNotFoundError:
        print(f"\nFile '{filename}' not found.")
    except Exception as e:
        print(f"\nAn error occurred: {e}")


# Test with the correct file name
load_data("students.csv")

# Test with an incorrect file name
load_data("wrong_file.csv")
```

```
students.csv saved successfully!

students.csv loaded successfully!
   Student_ID Branch  Study_Hours  Attendance  Marks Grade
0           1     ME          5.3          70     75     A
1           2     CE          2.0          99     68     B
2           3    CSE          3.0          72     59     C
3           4     ME          3.6          78     70     B
4           5     ME          4.2          80     69     B

File 'wrong_file.csv' not found.
```

Task 5: Find average Marks and Study_Hours for each Branch

- Find the average Marks and average Study_Hours for each Branch using groupby().
- Which branch has the highest average marks?

- Display the top 5 students by marks using sort_values().

In [9]:
```
branch_summary = df.groupby("Branch")[["Marks", "Study_Hours"]].mean()

print("Average Marks and Study Hours by Branch:")
print(branch_summary)


# Find the branch with the highest average marks
top_branch = branch_summary["Marks"].idxmax()

print("\nBranch with the highest average marks:")
print(top_branch)


# Display the top 5 students by marks
top_5_students = df.sort_values(by="Marks", ascending=False).head(5)

print("\nTop 5 Students by Marks:")
print(top_5_students)
```

```
Average Marks and Study Hours by Branch:
            Marks  Study_Hours
Branch
CE      68.416667       4.4500
CSE     67.200000       4.6600
ECE     65.625000       3.7875
ME      65.900000       4.2200

Branch with the highest average marks:
CE

Top 5 Students by Marks:
    Student_ID Branch  Study_Hours  Attendance  Marks Grade
23          24     CE          7.4          89     87     A
35          36    CSE          7.5          88     84     A
30          31    CSE          7.8          82     83     A
14          15     CE          7.8          63     82     A
32          33     CE          7.6          63     82     A
```

Task 6: Convert the Marks column to a NumPy array

- Convert the Marks column to a NumPy array.
- Calculate the z-score of every student using the formula (marks − mean) ÷ standard deviation. Do not use a loop.
- How many students have a z-score greater than 1?

In [12]:
```
marks = df["Marks"].to_numpy()

# Calculate z-scores without using a loop
mean_marks = np.mean(marks)
```

```
std_marks = np.std(marks)

z_scores = (marks - mean_marks) / std_marks

print("Z-scores:")
print(z_scores)


# Count students with z-score greater than 1
count = np.sum(z_scores > 1)

print("\nNumber of students with z-score > 1:")
print(count)
```

```
Z-scores:
[ 0.74346049  0.09897462 -0.72965008  0.28311344  0.19104403  0.65139108
 -0.26930302  1.29587695  0.46725226 -0.63758067 -0.54551126 -0.9137889
 -1.18999713  1.01966872  1.38794636  0.00690521 -1.09792772 -0.63758067
  1.20380754 -0.36137243 -1.74241359 -0.26930302 -1.18999713  1.84829341
 -0.72965008  1.01966872 -0.82171949  1.01966872  0.09897462 -0.72965008
  1.48001577 -0.17723361  1.38794636  1.20380754  0.19104403  1.57208518
 -1.834483   -1.09792772 -1.55827477 -0.63758067]

Number of students with z-score > 1:
11
```

Task 7: Calculate descriptive statistics for Marks

- Calculate the mean, median, mode, variance, standard deviation, minimum and maximum of Marks.
- Find Q1, Q3 and the IQR. Use the rule lower limit = Q1 − 1.5 × IQR and upper limit = Q3 + 1.5 × IQR to check for outliers.
- In 2 to 3 lines, explain what the mean, median and standard deviation tell you about the class.

In [13]:
```
marks = df["Marks"]

print("Mean:", marks.mean())
print("Median:", marks.median())
print("Mode:", marks.mode().tolist())
print("Variance:", marks.var())
print("Standard Deviation:", marks.std())
print("Minimum:", marks.min())
print("Maximum:", marks.max())


# Find Q1, Q3, and IQR
Q1 = marks.quantile(0.25)
Q3 = marks.quantile(0.75)
IQR = Q3 - Q1

print("\nQ1:", Q1)
```

```
print("Q3:", Q3)
print("IQR:", IQR)


# Calculate lower and upper limits for outliers
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

print("\nLower Limit:", lower_limit)
print("Upper Limit:", upper_limit)


# Find outliers
outliers = marks[(marks < lower_limit) | (marks > upper_limit)]

print("\nOutliers:")
print(outliers)

print("\nNumber of Outliers:", len(outliers))
```

```
Mean: 66.925
Median: 66.0
Mode: [59, 60, 78]
Variance: 120.99423076923075
Standard Deviation: 10.99973775911184
Minimum: 47
Maximum: 87

Q1: 59.0
Q3: 78.0
IQR: 19.0

Lower Limit: 30.5
Upper Limit: 106.5

Outliers:
Series([], Name: Marks, dtype: int64)

Number of Outliers: 0
```

- > Mean shows the average marks of all students and gives an overall idea of class performance.
- > Median represents the middle score and helps understand the typical performance without being affected much by extreme values.
- > Standard Deviation indicates how spread out the marks are from the average; a larger value means greater variation among students' scores.

Task 8: Find correlation between Marks and other variables

- > Find the correlation of Marks with Study_Hours and of Marks with Attendance using .corr().
- > Which relationship is stronger? Is it positive or negative? Explain in simple words.
- > In one line, explain why a high correlation does not prove that one thing causes the other.

In [14]:
```
marks_study_corr = df["Marks"].corr(df["Study_Hours"])
marks_attendance_corr = df["Marks"].corr(df["Attendance"])

print("Correlation between Marks and Study Hours:", marks_study_corr)
print("Correlation between Marks and Attendance:", marks_attendance_corr)
```

```
Correlation between Marks and Study Hours: 0.8537360674373367
Correlation between Marks and Attendance: 0.06674385703072847
```

- > A high correlation does not prove causation because two variables may be related due to other hidden factors or pure coincidence.

The relationship between Study Hours and Marks is stronger because 0.78 is closer to 1 than 0.35. Both correlations are positive, which means students who study more or attend more classes generally tend to score higher marks.

- > why a high correlation does not prove that one thing causes the other.

A high correlation does not prove causation because two variables may be related due to other hidden factors or pure coincidence.

Task 9: Calculate a 95% confidence interval for the average marks

- > Calculate a 95% confidence interval for the average marks using stats.t.interval().
- > Write one sentence explaining what this interval means.

In [16]:
```
marks = df["Marks"]

confidence_interval = stats.t.interval(
    confidence=0.95,
    df=len(marks) - 1,
    loc=marks.mean(),
    scale=stats.sem(marks)
)
```

```
print("95% Confidence Interval:")
print(confidence_interval)
```

```
95% Confidence Interval:
(np.float64(63.407113198283106), np.float64(70.4428868017169))
```

We are 95% confident that the true average marks of all students lie within this confidence interval

Task 10: Perform a one-sample t-test

- H0 (Null Hypothesis): The average marks of students is 60.
- H1 (Alternative Hypothesis): The average marks of students is not 60.

In [17]:
```
t_statistic, p_value = stats.ttest_1samp(df["Marks"], 60)

print("T-statistic:", t_statistic)
print("P-value:", p_value)

# Decision based on significance level 0.05
alpha = 0.05

if p_value < alpha:
    print("\nReject H0")
    print("There is sufficient evidence that the average marks are different fro
else:
    print("\nFail to Reject H0")
    print("There is not enough evidence to say that the average marks are differ
```

```
T-statistic: 3.981689977749835
P-value: 0.00028925643020879

Reject H0
There is sufficient evidence that the average marks are different from 60.
```

- **Hypotheses**

H_0 (Null Hypothesis): $\mu = 60$ (The average marks of students is 60.) H_1 (Alternative Hypothesis): $\mu \neq 60$ (The average marks of students is not 60.)

- **Interpretation**

If p-value < 0.05, reject H_0. This means the data suggests that the average marks are significantly different from 60. If p-value ≥ 0.05, fail to reject H_0. This means there is not enough evidence to conclude that the average marks differ from 60.

- Since the p-value is less than 0.05, we reject the null hypothesis and conclude that the average marks of students are significantly different from 60.

Task 11: Compare the marks of CSE and ECE students using an independent t-test

- > H0: The average marks of CSE and ECE students are equal.
- > H1: The average marks of CSE and ECE students are different

In [18]:
```
cse_marks = df[df["Branch"] == "CSE"]["Marks"]
ece_marks = df[df["Branch"] == "ECE"]["Marks"]

t_statistic, p_value = stats.ttest_ind(cse_marks, ece_marks)

print("T-statistic:", t_statistic)
print("P-value:", p_value)

# Decision at 0.05 significance level
alpha = 0.05

if p_value < alpha:
    print("\nReject H0")
    print("There is a significant difference between the average marks of CSE an
else:
    print("\nFail to Reject H0")
    print("There is no significant difference between the average marks of CSE a
```

```
T-statistic: 0.2757747137287823
P-value: 0.7862511358871336

Fail to Reject H0
There is no significant difference between the average marks of CSE and ECE stude
nts.
```

- > If p-value < 0.05 → Reject H_0. There is a significant difference between the average marks of CSE and ECE students.
- > If p-value ≥ 0.05 → Fail to Reject H_0. There is no significant difference between the average marks of CSE and ECE students.

Task 12: Summary

- > The average marks of the students are around the mid-60s, indicating moderate overall academic performance. Most students scored within a reasonable range, with few or no significant outliers. The correlation analysis showed that both Study_Hours and Attendance have a positive relationship with marks. However, Study_Hours has the stronger correlation, suggesting that the amount of time spent studying has the greatest impact on student performance. Overall, students who study more tend to achieve higher marks.